

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №6
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Унарные и бинарные операции над графами»

Выполнил:
студент группы 24ВВВ3
Майер В.С.
Савин А.В.

Приняли:
к.т.н., доцент Юрова О.В.
к.т.н., Деев М.В.

Пенза 2025

Цель работы – Изучить и практически освоить основные унарные и бинарные операции над графами, их свойства и визуальное представление результата их применения.

Лабораторное задание:

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) две матрицы M_1, M_2 смежности неориентированных помеченных графов G_1, G_2 . Выведите сгенерированные матрицы на экран.

Задание 2

1. Для матричной формы представления графов выполните операцию:

- а) отождествления вершин
- б) стягивания ребра
- в) расщепления вершины

Номера выбираемых для выполнения операции вершин ввести с клавиатуры.

Результат выполнения операции выведите на экран.

Задание 3

1. Для матричной формы представления графов выполните операцию:

- а) объединения $G = G_1 \cup G_2$
- б) пересечения $G = G_1 \cap G_2$
- в) кольцевой суммы $G = G_1 \oplus G_2$

Результат выполнения операции выведите на экран.

Результаты работы программы

```
C:\Users\Сергей\Desktop\lab66\x64\Debug\lab66.exe
1 - Генерация случайных графов
2 - Унарные операции над графом
3 - Бинарные операции над графами
0 - Выход
Выбор: 1
Введите размер матриц: 2

Матрица G1 (2x2):
1 0
0 1

Матрица G2 (2x2):
1 0
0 1

1 - Генерация случайных графов
2 - Унарные операции над графом
3 - Бинарные операции над графами
0 - Выход
Выбор:
```

Рисунок 1 - Результат работы программы (Генерация случайных графов)

```
1 - Генерация случайных графов
2 - Унарные операции над графом
3 - Бинарные операции над графами
0 - Выход
Выбор: 2
Введите размер графа: 3

Матрица исходная (3x3):
1 1 1
1 0 1
1 1 1

1 - Отождествление вершин
2 - Стягивание ребра
3 - Расщепление вершины
0 - Возврат в главное меню
Выбор операции: _
```

Рисунок 2 - Результат работы программы (Унарные операции над графом)

```
1 - Генерация случайных графов
2 - Унарные операции над графом
3 - Бинарные операции над графами
0 - Выход
Выбор: 3
Введите размер первой матрицы: 4
Введите размер второй матрицы: 2

Матрица G1 (4x4):
1 0 1 0
0 1 1 1
1 1 1 1
0 1 1 1

Матрица G2 (2x2):
1 0
0 1

1 - Объединение  $G = G1, G2$ 
2 - Пересечение  $G = G1, G2$ 
3 - Кольцевая сумма  $G = G1, G2$ 
0 - Возврат в главное меню
Выбор операции: █
```

Рисунок 3 - Результат работы программы (Бинарные операции над графами)

Вывод: Изучили и практически освоили основные унарные и бинарные операции над графами, их свойства и визуальное представление результата их применения.

Приложение А

Код программы:

```
#define _CRT_SECURE_NO_WARNINGS

#include <stdio.h>

#include <stdlib.h>

#include <time.h>

#include <locale.h>

void generateSymmetricMatrix(int** matrix, int size) {

    for (int i = 0; i < size; i++) {

        for (int j = i; j < size; j++) {

            if (i == j) {

                matrix[i][j] = rand() % 2;

            }

            else {

                int random_value = rand() % 2;

                matrix[i][j] = random_value;

                matrix[j][i] = random_value;

            }

        }

    }

}

void generateDifferentSizedMatrices(int*** M1, int*** M2, int* size1, int* size2) {

    printf("Введите размер первой матрицы: ");

    scanf("%d", size1);

    printf("Введите размер второй матрицы: ");

    scanf("%d", size2);

    printf("\n");

    *M1 = (int**)malloc(*size1 * sizeof(int*));

    for (int i = 0; i < *size1; i++) {

        (*M1)[i] = (int*)malloc(*size1 * sizeof(int));

    }

}
```

```

    }

    *M2 = (int**)malloc(*size2 * sizeof(int*));

    for (int i = 0; i < *size2; i++) {
        (*M2)[i] = (int*)malloc(*size2 * sizeof(int));
    }

    generateSymmetricMatrix(*M1, *size1);
    generateSymmetricMatrix(*M2, *size2);
}

void printMatrix(int** matrix, int size, const char* name) {
    printf("Матрица %s (%dx%d):\n", name, size, size);

    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++) {
            printf("%d ", matrix[i][j]);
        }

        printf("\n");
    }

    printf("\n");
}

int** copyMatrix(int** source, int size) {
    int** copy = (int**)malloc(size * sizeof(int*));

    for (int i = 0; i < size; i++) {
        copy[i] = (int*)malloc(size * sizeof(int));

        for (int j = 0; j < size; j++) {
            copy[i][j] = source[i][j];
        }
    }

    return copy;
}

void freeMatrix(int** matrix, int size) {

```

```

    if (matrix != NULL) {
        for (int i = 0; i < size; i++) {
            if (matrix[i] != NULL) {
                free(matrix[i]);
            }
        }
        free(matrix);
    }
}

int** identifyVertices(int** matrix, int size, int u, int v, int* new_size) {
    if (u < 0 || u >= size || v < 0 || v >= size || u == v) {
        printf("Ошибка: Неверные вершины для отождествления!\n");
        *new_size = size;
        return copyMatrix(matrix, size);
    }

    *new_size = size - 1;

    int** result = (int**)malloc(*new_size * sizeof(int*));

    for (int i = 0; i < *new_size; i++) {
        result[i] = (int*)malloc(*new_size * sizeof(int));
    }

    int new_vertex = (u < v) ? u : v;

    for (int i = 0, new_i = 0; i < size; i++) {
        if (i == v) continue;

        for (int j = 0, new_j = 0; j < size; j++) {
            if (j == v) continue;

            if (i == u) {
                if (j == u) {
                    result[new_i][new_j] = matrix[u][u] || matrix[v][v];
                }
            }
        }
    }
}

```

```

        else {
            result[new_i][new_j] = matrix[u][j] || matrix[v][j];
        }
    }
    else if (j == u) {
        result[new_i][new_j] = matrix[i][u] || matrix[i][v];
    }
    else {
        result[new_i][new_j] = matrix[i][j];
    }
    new_j++;
}
new_i++;
}

return result;
}

int** contractEdge(int** matrix, int size, int u, int v, int* new_size) {
    if (u < 0 || u >= size || v < 0 || v >= size || u == v) {
        printf("Ошибка: Неверные вершины для стягивания!\n");
        *new_size = size;
        return copyMatrix(matrix, size);
    }

    if (matrix[u][v] == 0) {
        printf("Ошибка: Ребро между вершинами %d и %d не существует!\n", u, v);
        *new_size = size;
        return copyMatrix(matrix, size);
    }

    return identifyVertices(matrix, size, u, v, new_size);
}

```

```

int** splitVertex(int** matrix, int size, int v, int* new_size) {

    if (v < 0 || v >= size) {

        printf("Ошибка: Неверная вершина для расщепления!\n");

        *new_size = size;

        return copyMatrix(matrix, size);

    }

    *new_size = size + 1;

    int** result = (int**)malloc(*new_size * sizeof(int*));

    for (int i = 0; i < *new_size; i++) {

        result[i] = (int*)malloc(*new_size * sizeof(int));

    }

    for (int i = 0; i < size; i++) {

        for (int j = 0; j < size; j++) {

            result[i][j] = matrix[i][j];

        }

    }

    for (int i = 0; i < *new_size; i++) {

        result[i][*new_size - 1] = 0;

        result[*new_size - 1][i] = 0;

    }

    result[*new_size - 1][*new_size - 1] = 0;

    result[v][*new_size - 1] = 1;

    result[*new_size - 1][v] = 1;

    for (int i = 0; i < size; i++) {

        if (matrix[v][i] == 1 && i != v) {

            if (rand() % 2 == 0) {

                result[*new_size - 1][i] = 1;

                result[i][*new_size - 1] = 1;

                result[v][i] = 0;

            }

        }

    }

}

```

```

        result[i][v] = 0;

    }

}

return result;

}

int** unionGraphs(int** M1, int size1, int** M2, int size2, int* result_size) {

    *result_size = (size1 > size2) ? size1 : size2;

    int** result = (int**)malloc(*result_size * sizeof(int*));

    for (int i = 0; i < *result_size; i++) {

        result[i] = (int*)malloc(*result_size * sizeof(int));

        for (int j = 0; j < *result_size; j++) {

            int val1 = (i < size1 && j < size1) ? M1[i][j] : 0;

            int val2 = (i < size2 && j < size2) ? M2[i][j] : 0;

            result[i][j] = val1 || val2;

        }

    }

    return result;

}

int** intersectionGraphs(int** M1, int size1, int** M2, int size2, int* result_size) {

    *result_size = (size1 < size2) ? size1 : size2;

    int** result = (int**)malloc(*result_size * sizeof(int*));

    for (int i = 0; i < *result_size; i++) {

        result[i] = (int*)malloc(*result_size * sizeof(int));

        for (int j = 0; j < *result_size; j++) {

            int val1 = M1[i][j];

            int val2 = (i < size2 && j < size2) ? M2[i][j] : 0;

            result[i][j] = val1 && val2;

        }

    }

}

```

```

    }

    return result;
}

int** ringSumGraphs(int** M1, int size1, int** M2, int size2, int* result_size) {
    *result_size = (size1 > size2) ? size1 : size2;

    int** result = (int**)malloc(*result_size * sizeof(int*));

    for (int i = 0; i < *result_size; i++) {
        result[i] = (int*)malloc(*result_size * sizeof(int));

        for (int j = 0; j < *result_size; j++) {
            int val1 = (i < size1 && j < size1) ? M1[i][j] : 0;
            int val2 = (i < size2 && j < size2) ? M2[i][j] : 0;
            result[i][j] = val1 ^ val2;
        }
    }

    return result;
}

int main() {
    setlocale(LC_ALL, "");
    srand(time(NULL));

    int choice, size, u, v;

    while (1) {
        printf("1 - Генерация случайных графов\n");
        printf("2 - Унарные операции над графом\n");
        printf("3 - Бинарные операции над графами\n");
        printf("0 - Выход\n");
        printf("Выбор: ");
        scanf("%d", &choice);

        if (choice == 0) break;
    }
}

```

```

switch (choice) {
case 1: {
    printf("Введите размер матриц: ");
    scanf("%d", &size);
    printf("\n");

    int** M1 = (int**)malloc(size * sizeof(int*));
    int** M2 = (int**)malloc(size * sizeof(int*));
    for (int i = 0; i < size; i++) {
        M1[i] = (int*)malloc(size * sizeof(int));
        M2[i] = (int*)malloc(size * sizeof(int));
    }

    generateSymmetricMatrix(M1, size);
    generateSymmetricMatrix(M2, size);

    printMatrix(M1, size, "G1");
    printMatrix(M2, size, "G2");

    freeMatrix(M1, size);
    freeMatrix(M2, size);
    break;
}

case 2: {
    printf("Введите размер графа: ");
    scanf("%d", &size);
    printf("\n");

    int** matrix = (int**)malloc(size * sizeof(int*));
    for (int i = 0; i < size; i++) {
        matrix[i] = (int*)malloc(size * sizeof(int));
    }

    generateSymmetricMatrix(matrix, size);

```

```

printMatrix(matrix, size, "исходная");

int op_choice;

do {

    printf("1 - Отождествление вершин\n");

    printf("2 - Стягивание ребра\n");

    printf("3 - Расщепление вершины\n");

    printf("0 - Возврат в главное меню\n");

    printf("Выбор операции: ");

    scanf("%d", &op_choice);

    printf("\n");

    if (op_choice == 0) break;

    int new_size;

    int** result = NULL;

    switch (op_choice) {

    case 1:

        printf("Введите номера вершин для отождествления (0-%d): ", size -
1);

        scanf("%d %d", &u, &v);

        if (u >= 0 && u < size && v >= 0 && v < size && u != v) {

            result = identifyVertices(matrix, size, u, v, &new_size);

            printMatrix(result, new_size, "после отождествления вершин");

        }

        else {

            printf("Ошибка: Неверные номера вершин!\n");

        }

        break;

    case 2:

        printf("Введите номера вершин ребра для стягивания (0-%d): ", size
- 1);

        scanf("%d %d", &u, &v);

```

```

        if (u >= 0 && u < size && v >= 0 && v < size && u != v) {
            result = contractEdge(matrix, size, u, v, &new_size);
            printMatrix(result, new_size, "после стягивания ребра");
        }
        else {
            printf("Ошибка: Неверные номера вершин!\n");
        }
        break;
    case 3:
        printf("Введите номер вершины для расщепления (0-%d): ", size -
1);

        scanf("%d", &v);
        if (v >= 0 && v < size) {
            result = splitVertex(matrix, size, v, &new_size);
            printMatrix(result, new_size, "после расщепления вершины");
        }
        else {
            printf("Ошибка: Неверный номер вершины!\n");
        }
        break;
    default:
        printf("Неверный выбор операции!\n");
        continue;
    }

    if (result != NULL) {
        freeMatrix(result, new_size);
    }

} while (1);

freeMatrix(matrix, size);

break;
}

```

```

case 3: {

    int size1, size2;

    int** M1 = NULL;

    int** M2 = NULL;

    generateDifferentSizedMatrices(&M1, &M2, &size1, &size2);

    printMatrix(M1, size1, "G1");

    printMatrix(M2, size2, "G2");

    int op_choice;

    do {

        printf("1 - Объединение G = G1, G2\n");

        printf("2 - Пересечение G = G1, G2\n");

        printf("3 - Кольцевая сумма G = G1, G2\n");

        printf("0 - Возврат в главное меню\n");

        printf("Выбор операции: ");

        scanf("%d", &op_choice);

        printf("\n");

        if (op_choice == 0) break;

        int** result = NULL;

        int result_size;

        switch (op_choice) {

            case 1:

                result = unionGraphs(M1, size1, M2, size2, &result_size);

                printMatrix(result, result_size, "объединения G1, G2");

                break;

            case 2:

                result = intersectionGraphs(M1, size1, M2, size2, &result_size);

                printMatrix(result, result_size, "пересечения G1, G2");

```

```

        break;

    case 3:

        result = ringSumGraphs(M1, size1, M2, size2, &result_size);

        printMatrix(result, result_size, "кольцевой суммы G1, G2");

        break;

    default:

        printf("Неверный выбор операции!\n");

        continue;

    }

    if (result != NULL) {

        freeMatrix(result, result_size);

    }

} while (1);

freeMatrix(M1, size1);

freeMatrix(M2, size2);

break;

}

default:

    printf("Неверный выбор задания!\n");

    break;

}

}

printf("Программа завершена.\n");

return 0;

}

```